

Manual de Sintaxis

MOTORINFERENCIA

Basado en `motor_inferencia.1` y `motor_inferencia.y`

Rodolfo Rodrigo

Departamento de Electromecánica, Facultad de Ingeniería, UNSJ

Instituto de Automática (INAUT), UNSJ-CONICET

Marzo 2026

Índice

1. Introducción	2
2. Reglas Léxicas	2
2.1. Expresiones regulares de los tokens	2
2.2. Funciones builtin reconocidas como tokens	3
2.3. Operadores Unicode	3
2.4. Notación científica	3
3. Gramática Formal (BNF)	3
3.1. Producciones	3
3.2. Conjuntos e intervalos	4
3.3. Llamadas a función	5
4. Precedencia y Asociatividad (Binding Powers)	5
5. Sistema de Tipos	5
6. Conjuntos Discretos	6
7. Intervalos	6
8. Reglas Semánticas del Pipeline	7
9. Formato de Entrada JSON	7
9.1. Ejemplo completo	8
10. Diferencias con MotorEvaluación	9
11. Referencia Rápida	9

1. Introducción

Este manual describe la sintaxis formal del lenguaje de restricciones (*constraints*) aceptado por MOTORINFERENCIA, un pipeline modular de propagación para problemas de *Constraint Satisfaction* (CSP).

A diferencia de MOTOREVALUACIÓN que evalúa expresiones sobre valores concretos, MOTORINFERENCIA opera sobre *dominios*: intervalos $[lo, hi]$ para variables numéricas, listas explícitas para enteros, y subconjuntos para booleanos y sets. Las restricciones escritas en este lenguaje se propagan mediante el algoritmo AC-3 para reducir los dominios a un punto fijo consistente.

El parser interno utiliza el algoritmo de Pratt (Top-Down Operator Precedence, Vaughan Pratt 1973) con 9 niveles de *binding power*.

2. Reglas Léxicas

El analizador léxico (definido en `motor_inferencia.1`) opera en modo *case-insensitive*. Soporta operadores Unicode UTF-8 como alternativas a los operadores ASCII.

2.1. Expresiones regulares de los tokens

Token	Patrón (regex)	Descripción
NUMBER	<code>[0-9]+(" "[0-9]+)?([eE] [+−]?[0-9]+)?</code>	Número (entero, real, científico)
NUMBER	<code>INF INFINITY \xe2\x88\x9e</code>	Infinito positivo ($+\infty$)
BOOLEAN	<code>true false</code>	Literal booleano
IDENTIFIER	<code>[a-zA-Z_] [a-zA-Z_0-9]*</code>	Variable, etiqueta o función
AND	<code>AND</code>	Conjunción
OR	<code>OR</code>	Disyunción
NOT	<code>NOT</code>	Negación
IN	<code>IN \xe2\x88\x88</code>	Pertenencia ($\in$)
IMPLICA	<code>IMPLICA</code>	Implicación lógica
GTE	<code>>= \xe2\x89\xa5</code>	$\geq$
LTE	<code><= \xe2\x89\xa4</code>	$\leq$
NEQ	<code><> != \xe2\x89\xa0</code>	$\neq$
EQ	<code>= ==</code>	$=$
GT	<code>></code>	$>$
LT	<code><</code>	$<$
PLUS	<code>+</code>	Suma
MINUS	<code>-</code>	Resta
TIMES	<code>*</code>	Multiplicación
DIVIDE	<code>/</code>	División
MOD	<code>%</code>	Módulo
POWER	<code>^ **</code>	Potencia
LPAREN	<code>(</code>	Paréntesis izq.
RPAREN	<code>)</code>	Paréntesis der.
LBRACKET	<code>[</code>	Corchete izq.
RBRACKET	<code>]</code>	Corchete der.
LCURLY	<code>{</code>	Llave izq.
RCURLY	<code>}</code>	Llave der.

Token	Patrón (regex)	Descripción
COMMA	,	Coma

2.2. Funciones builtin reconocidas como tokens

Función	Dominio	Retorno	Descripción
$\text{abs}(x)$	numeric	numeric	Valor absoluto
$\text{sqr}(x)$	numeric	numeric	Cuadrado (x^2)
$\text{sqrt}(x)$	numeric	numeric	Raíz cuadrada
$\text{sin}(x)$	numeric	numeric	Seno
$\text{cos}(x)$	numeric	numeric	Coseno
$\text{tan}(x)$	numeric	numeric	Tangente
$\text{ln}(x)$	numeric	numeric	Logaritmo natural
$\text{exp}(x)$	numeric	numeric	Exponencial
$\text{log}(x)$	numeric	numeric	Logaritmo base 10
$\text{interval}(a, b)$	numeric ²	numeric	Construye intervalo $[a, b]$
$\text{complex}(r, i)$	numeric ²	complex	Construye número complejo
$\text{cardinality}(S)$	set	integer	Cardinalidad $ S $

2.3. Operadores Unicode

MOTORINFERENCIA reconoce secuencias UTF-8 de 3 bytes como alternativas a los operadores ASCII. Esto permite escribir restricciones con notación matemática directa:

Símbolo	Code point	Bytes UTF-8	Equivalente ASCII
$\in$	U+2208	E2 88 88	IN
$\neq$	U+2260	E2 89 A0	<> o !=
$\leq$	U+2264	E2 89 A4	<=
$\geq$	U+2265	E2 89 A5	>=
∞	U+221E	E2 88 9E	INF

2.4. Notación científica

Los números pueden usar notación exponencial con **e** o **E**:

$$1.5\text{e}-3 = 0,0015, \quad 2.998\text{E}8 = 2,998 \times 10^8, \quad 6\text{e}23 = 6 \times 10^{23}$$

3. Gramática Formal (BNF)

La gramática se presenta en notación BNF extendida. Corresponde a las funciones NUD (null denotation), LED (left denotation) y ParseSetOrInterval del Pratt parser en `PrattParser.pas`.

3.1. Producciones

expression ::= *or-expr*

or-expr ::= *and-expr*
| *or-expr* OR *and-expr*

and-expr ::= *not-expr*

		<i>and-expr</i>	AND	<i>not-expr</i>
<i>not-expr</i>	::=	<i>in-expr</i>		
		NOT		<i>not-expr</i>
<i>in-expr</i>	::=	<i>comp-expr</i>		
		<i>comp-expr</i>	IN	<i>set-or-interval</i>
<i>comp-expr</i>	::=	<i>add-expr</i>		
		<i>comp-expr</i>	=	<i>add-expr</i>
		<i>comp-expr</i>	<>	<i>add-expr</i>
		<i>comp-expr</i>	<	<i>add-expr</i>
		<i>comp-expr</i>	>	<i>add-expr</i>
		<i>comp-expr</i>	<=	<i>add-expr</i>
		<i>comp-expr</i>	>=	<i>add-expr</i>
<i>add-expr</i>	::=	<i>mult-expr</i>		
		<i>add-expr</i>	+	<i>mult-expr</i>
		<i>add-expr</i>	-	<i>mult-expr</i>
<i>mult-expr</i>	::=	<i>power-expr</i>		
		<i>mult-expr</i>	*	<i>power-expr</i>
		<i>mult-expr</i>	/	<i>power-expr</i>
		<i>mult-expr</i>	%	<i>power-expr</i>
<i>power-expr</i>	::=	<i>unary-expr</i>		
		<i>unary-expr</i>	^	<i>power-expr</i>
<i>unary-expr</i>	::=	<i>primary</i>		
		-		<i>unary-expr</i>
<i>primary</i>	::=	NUMBER		
		IDENTIFIER		
		BOOLEAN		
		<i>func-call</i>		
		(	<i>expression</i>	)

3.2. Conjuntos e intervalos

<i>set-or-interval</i>	::=	<i>discrete-set</i>
		<i>interval</i>
		IDENTIFIER
<i>discrete-set</i>	::=	{ <i>set-elements</i> }
		{ }
<i>set-elements</i>	::=	<i>expression</i>
		<i>set-elements</i> , <i>expression</i>
<i>interval</i>	::=	[<i>expression</i> , <i>expression</i>]
		(<i>expression</i> , <i>expression</i>)
		[<i>expression</i> , <i>expression</i>)

| (*expression* , *expression*]

3.3. Llamadas a función

```
func-call ::= BUILTIN ( arg-list )
          | interval ( expr , expr )
          | complex ( expr , expr )
          | IDENTIFIER ( arg-list )
          | IDENTIFIER ( )

arg-list ::= expression
          | arg-list , expression
```

Donde **BUILTIN** $\in \{\text{abs, sqr, sqrt, sin, cos, tan, ln, exp, log, cardinality}\}$.

4. Precedencia y Asociatividad (Binding Powers)

La tabla de precedencia se deriva directamente de los *binding powers* (BP) del Pratt parser en `PrattParser.pas`. Un BP mayor significa mayor precedencia (el operador “ata” más fuerte):

BP	Operadores	Tipo	Asociatividad	Producción
10	OR	Infijo	Izquierda	<i>or-expr</i>
20	AND	Infijo	Izquierda	<i>and-expr</i>
25	NOT	Prefijo	Derecha	<i>not-expr</i>
40	IN	Infijo	Izquierda	<i>in-expr</i>
50	= <><><= >=	Infijo	No asoc.	<i>comp-expr</i>
60	+ -	Infijo	Izquierda	<i>add-expr</i>
70	* / %	Infijo	Izquierda	<i>mult-expr</i>
80	^ **	Infijo	Derecha	<i>power-expr</i>
90	- (unario)	Prefijo	Derecha	<i>unary-expr</i>

Potencia right-associative. La potencia se asocia a la derecha, lo que significa:

$$2 \wedge 3 \wedge 4 \equiv 2 \wedge (3 \wedge 4) = 2^{3^4} = 2^{81}$$

Esto se implementa en el Pratt parser llamando a `Expr(79)` en el LED de $\wedge$, es decir con $rbp = bp - 1$.

NOT vs. AND/OR. NOT tiene BP=25, entre AND (20) y IN (40). Esto significa que NOT captura una comparación completa pero no atraviesa AND:

$$\text{NOT } x > 5 \text{ AND } y < 3 \equiv (\text{NOT } (x > 5)) \text{ AND } (y < 3)$$

5. Sistema de Tipos

MOTORINFERENCIA soporta cuatro tipos de variables, cada uno con un modelo de dominio diferente:

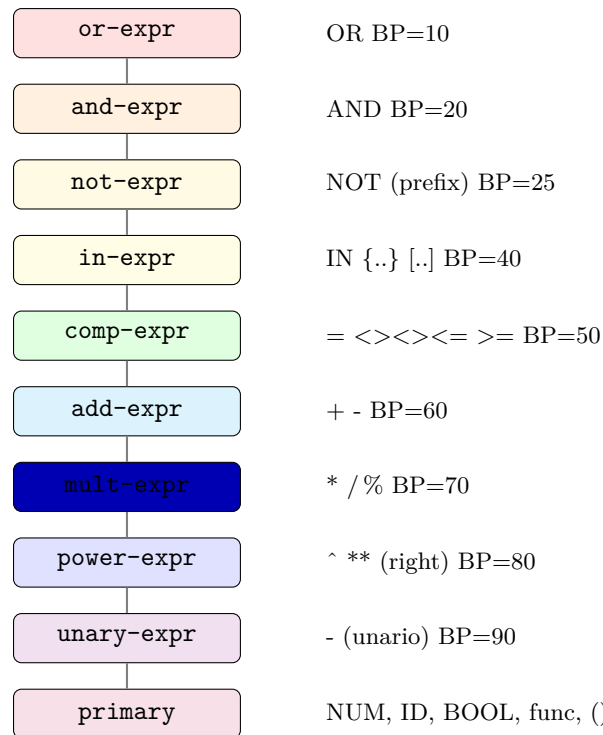


Figura 1: Jerarquía de binding powers del Pratt parser.

Tipo	Dominio	Valor	Ejemplo
boolean	[true, false]	Subconjunto del dominio	value: [true]
set	Lista de etiquetas	Subconjunto del dominio	domain: [."A", "B", "C"]
integer	Lista explícita de enteros	Subconjunto del dominio	domain: [1,2,3,4,5]
numeric	Intervalo [lo, hi]	Subintervalo [lo', hi']	domain: [0.0, 100.0]

Diferencia clave con MotorEvaluación. En MotorEvaluación, una variable tiene un único valor concreto ("value": 42). En MOTORINFERENCIA, una variable tiene un *dominio* (conjunto de valores posibles) y un *valor actual* (subconjunto del dominio que se reduce durante la propagación).

6. Conjuntos Discretos

Los conjuntos discretos se escriben entre llaves con etiquetas separadas por comas:

```
estado IN {ACTIVO, PAUSADO, DETENIDO}
```

Las etiquetas son identificadores (sin comillas en la expresión; con comillas en el JSON de declaración). El conjunto vacío se escribe {}.

También se puede usar una variable de tipo set directamente:

```
fiebre IN sintomas
```

7. Intervalos

Los intervalos numéricos soportan cuatro combinaciones de extremos abiertos y cerrados, y admiten $\pm\infty$ como límites:

Sintaxis	Significado matemático	Ejemplo
$x \text{ IN } [a, b]$	$a \leq x \leq b$	temp IN [36.0, 38.5]
$x \text{ IN } (a, b)$	$a < x < b$	ph IN (6.5, 7.5)
$x \text{ IN } [a, b)$	$a \leq x < b$	hora IN [0, 24)
$x \text{ IN } (a, b]$	$a < x \leq b$	edad IN (0, 120]
$x \text{ IN } [0, \text{INF})$	$x \geq 0$	masa IN [0, INF)
$x \text{ IN } (-\text{INF}, 0]$	$x \leq 0$	delta IN (-INF, 0]

Los límites a y b pueden ser subexpresiones aritméticas arbitrarias:

$$x \text{ IN } [y - 0.5, y + 0.5]$$

8. Reglas Semánticas del Pipeline

Además de la gramática libre de contexto (que define la sintaxis pura), MOTORINFERENCIA aplica 9 reglas semánticas en la etapa de **SyntaxChecker**. Estas reglas se validan *después* del parsing y se expresan formalmente como sigue:

Regla	Definición formal
R0	$\forall \text{campo} \in \{\text{variables}, \text{expressions}\} : \text{campo} \in \text{JSON} \wedge \text{tipo}(\text{campo}) = \text{array}$
R1	$\forall v \in \text{DeclaredVars} : \exists e \in \text{Expressions} : v.\text{name} \in \text{freeVars}(e)$
R2	$\forall f \in \text{DeclaredFuncs} : \exists e \in \text{Expressions} : f.\text{name} \in \text{calledFuncs}(e)$
R3	$\forall e \in \text{Expressions}, \forall v \in \text{freeVars}(e) : v \in \text{DeclaredVars} \vee v \in \text{Literals}$
R4	$\forall e \in \text{Expressions}, \forall f \in \text{calledFuncs}(e) : f \in \text{DeclaredFuncs} \vee f \in \text{BuiltinFuncs}$
R5	$\forall c \in \text{Constraints} : \text{Parse}(c) \neq \perp$ (parseabilidad sintáctica)
R6	$\forall v \in \text{DeclaredVars} : v.\text{value} \subseteq v.\text{domain}$
R7	$\forall \text{call}(f, \text{args}) : \text{args} = f.\text{inputs} $ (si f tiene aridad fija)
R8	$\forall f \in \text{DeclaredFuncs} : f.\text{name} \notin \text{BuiltinFuncs}$ (no override)

Donde: $\text{BuiltinFuncs} = \{\text{abs}, \text{sqr}, \text{sqrt}, \text{sin}, \text{cos}, \text{tan}, \text{ln}, \text{exp}, \text{log}, \text{interval}, \text{complex}, \text{cardinality}\}$.

Interpretación práctica. **R1** y **R2** detectan declaraciones huérfanas (declaradas pero no usadas). **R3** y **R4** detectan referencias a entidades no declaradas. **R5** valida que cada constraint se pueda parsear con la gramática de este manual. **R6** verifica consistencia dominio/valor. **R7** controla aridad de funciones. **R8** impide redeclarar builtins.

9. Formato de Entrada JSON

Listing 1: Estructura JSON de entrada

```

1 {
2   "variables": [
3     {"name": "x", "type": "numeric",
```

```

4      "domain": [lo, hi], "value": [lo', hi']},
5      {"name": "estado", "type": "set",
6        "domain": ["A","B","C"], "value": ["A","B"]},
7      {"name": "n", "type": "integer",
8        "domain": [1,2,3,4,5], "value": [2,3,4]},
9      {"name": "activo", "type": "boolean",
10       "domain": [true,false], "value": [true]}
11 ],
12 "expressions": [
13   {"constraints": ["expr1", "expr2", ...]}
14 ],
15 "functions": [
16   {"name": "mifunc",
17     "inputs": [{"name": "x", "type": "numeric", "domain": [0,100]}],
18     "outputs": [{"name": "r", "type": "numeric", "domain": [0,10]}]}
19 ]
20 }

```

9.1. Ejemplo completo

Listing 2: Ejemplo: control de proceso industrial

```

1 {
2   "variables": [
3     {"name": "temperatura", "type": "numeric",
4       "domain": [0.0, 500.0], "value": [150.0, 250.0]},
5     {"name": "presion", "type": "set",
6       "domain": ["ALTA","MEDIA","BAJA"],
7       "value": ["ALTA","MEDIA"]},
8     {"name": "velocidad", "type": "numeric",
9       "domain": [0.0, 100.0], "value": [0.0, 100.0]},
10    {"name": "activo", "type": "boolean",
11      "domain": [true, false], "value": [true]}
12  ],
13  "expressions": [
14    {"constraints": [
15      "temperatura IN [100.0, 300.0]",
16      "presion IN {ALTA, MEDIA}",
17      "activo AND velocidad > 10.0",
18      "NOT (temperatura > 400.0)",
19      "sqrt(velocidad^2 + 1.0) < 50.0",
20      "temperatura * 0.01 + velocidad <= 100.0"
21    ]}
22  ]
23 }

```


10. Diferencias con MotorEvaluación

Característica	MotorEvaluación	MotorInferencia
Parser	Recursivo descendente	Pratt (top-down operator prec.)
Niveles de prec.	6	9 (con binding powers explícitos)
Potencia	No soportada	$\wedge$ y ** (right-assoc)
Menos unario	No soportado	BP=90
Módulo	Solo mod	mod y %
NOT vs. AND	NOT tiene la mayor prec.	NOT (BP=25) entre AND y IN
Conjuntos	Solo IN variable	IN {...} discretos
Intervalos	Expandidos a AND de comparaciones	Nodo ntInterval nativo
Unicode	No	$\in, \neq, \leq, \geq, \infty$
Not. científ.	No	Sí (1.5e-3)
∞	No	Sí (INF/INFINITY)
IMPLICA	No	Sí
Funciones	15 (incl. arctan , ann)	12 (incl. log , interval , complex)
!=	No	Sí (alternativa a <>)
Reglas sem.	Validación de tipos inline	9 reglas formales (R0–R8)

11. Referencia Rápida

Restricciones válidas (ejemplos):

- | | |
|---|---------------------------------|
| ▪ <code>temperatura >20.0</code> | comparación simple |
| ▪ <code>presion IN {ALTA, MEDIA, BAJA}</code> | pertenencia a conjunto discreto |
| ▪ <code>x IN [0.0, 100.0]</code> | pertenencia a intervalo cerrado |
| ▪ <code>y IN (-INF, 0.0]</code> | semi-intervalo con infinito |
| ▪ <code>activo AND NOT (x >50)</code> | lógica con negación |
| ▪ <code>2^3^2 = 2³² = 512</code> | potencia right-assoc |
| ▪ <code>sqrt(x^2 + y^2) <= 10.0</code> | funciones anidadas |
| ▪ <code>x% 3 = 0</code> | módulo con % |
| ▪ <code>-x + 2*y >= 0</code> | menos unario |
| ▪ <code>1.5e-3 * velocidad <0.1</code> | notación científica |
| ▪ <code>mifunc(x, y) >umbral</code> | función user-defined |